

Meissel Lehmer

Meissel–Lehmer 算法

「Meissel–Lehmer 算法」是一种能在亚线性时间复杂度内求出 $\pi(x)$ 内质数个数的一种算法。

记号规定

- 表示对 x 下取整得到的结果。
- 表示第 n 个质数， p_n 。
- 表示 x 范围内素数的个数。
- 表示莫比乌斯函数。
- 对于集合 S ， $|S|$ 表示集合 S 的大小。
- 表示 x 最小的质因子。
- 表示 x 最大的质因子。

Meissel–Lehmer 算法求 $\pi(x)$

定义 $\pi(x)$ 为所有小于 x 的正整数中满足其所有质因子都大于 y 的数的个数，即：

再定义 $\pi_k(x, y)$ 表示为所有小于 x 的正整数中满足可重质因子恰好有 k 个且所有质因子都大于 y 的数的个数，即：

特殊的，我们定义： $\pi_0(x, y) = \pi(x)$ ，如此便有：

这个无限和式实际上是可以表示为有限和式的，因为在 $y \leq \sqrt{x}$ 时，有 $\pi_k(x, y) = 0$ 。

设 z 为满足 $z^2 \leq x$ 的整数，再记 $y = p_z$ 。

在 y 时，有 $\pi_k(x, y) = \pi_k(x, p_z)$ ，由此我们可以推出：

这样，计算 $\pi(x)$ 便可以转化为计算 $\pi_0(x, p_z)$ 与 $\pi_k(x, p_z)$ 。

计算 $\pi_2(x, a)$

由等式 $\pi_2(x, a) = \pi_2(x, p_a)$ 我们可以得出 $\pi_2(x, a)$ 等于满足 $p_a < p < x/p$ 且 p 的质数对 (p, q) 的个数。

首先我们注意到 $\pi_2(x, a) = \pi_2(x, p_a)$ 。此外，对于每个 p ，我们都有 $\pi_2(x, p) = \pi_2(x, p_a)$ 。因此：

当 $p > \sqrt{x}$ 时，我们有 $\pi_2(x, p) = 0$ 。因此，我们可以筛区间 $[p_a, \sqrt{x}]$ ，然后对于所有的质数 p 计算 $\pi_2(x, p)$ 。为了减少上述算法的空间复杂度，我们可以考虑分块，块长为 $\sqrt{x}$ 。若块长 $\sqrt{x}$ ，则我们可以在 $O(\sqrt{x})$ 的时间复杂度， $O(\sqrt{x})$ 的空间复杂度内计算。

计算 $\phi(x, a)$

对于 n ，考虑所有不超过 n 的正整数，满足它的所有质因子都大于 $\sqrt{n}$ 。这些数可以被分为两类：

1. 可以被 $\sqrt{n}$ 整除的；
2. 不可以被 $\sqrt{n}$ 整除的。

属于第 1 类的数有 $\sqrt{n}$ 个，属于第二类的数有 $\sqrt{n}$ 个。

因此我们得出结论：

定理： 函数 $\phi(n)$ 满足下列性质

计算 $\phi(n)$ 的简单方法可以从这个定理推导出来：我们重复使用等式 $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$ ，知道最后得到 $\phi(n)$ 。这个过程可以看作从根节点 n 开始创建有根二叉树，图 1 画出了这一过程。通过这种方法，我们得到如下公式：

上图表示计算 $\phi(n)$ 过程的二叉树：叶子节点权值之和就是 $\phi(n)$ 。

但是，这样需要计算太多东西。因为 n ，仅仅计算为 n 个不超过 $\sqrt{n}$ 质数的乘积的数，如果按照这个方法计算，会有至少 $\sqrt{n}$ 个项，没有办法我们对复杂度的需求。

为了限制这个二叉树的「生长」，我们要改变原来的终止条件。这是原来的终止条件。

终止条件： 如果 $n < \sqrt{n}$ ，则不要再对节点 n 调用等式 $\phi(n)$ 。

我们把它改成更强的终止条件：

终止条件： 如果满足下面 k 个条件中的一个，不要再对节点 n 调用等式 $\phi(n)$ ：

1. $n < \sqrt{n}$ ；
2. n 是质数。

我们根据 **终止条件** 将原二叉树上的叶子分成两种：

1. 如果叶子节点 n 满足 $n < \sqrt{n}$ ，则称这种叶子节点为 **普通叶子**；
2. 如果叶子节点 n 满足 n 是质数且 $n > \sqrt{n}$ ，则称这种节点为 **特殊叶子**。

由此我们得出：

定理： 我们有：

其中 S_1 表示 **普通叶子** 的贡献：

S_2 表示 **特殊叶子** 的贡献：

计算 $\phi(n)$ 显然是可以在 $O(\sqrt{n})$ 的时间复杂度内解决的，现在我们要考虑如何计算 S 。

计算 S

我们有：

我们将这个等式改写为：

其中：

注意到计算 S_1 的和式中涉及到的 p 都是质数，证明如下：

如果不是这样，因为有 $p \mid S_1$ ，所以有 $p \mid S_1$ ，这与 S_1 矛盾，所以原命题成立。

更多的，当 $p \mid S_1$ 时，有 $p \mid S_1$ 。因此我们有：

计算 S_1

因为：

所以：

所以计算 S_1 的和式中的项都是 p 。所以我们实际上要计算质数对 (p, q) 的个数，满足： $p \mid S_1$ 。

因此：

有了这个等式我们便可以在 $O(\sqrt{N})$ 的时间内计算 S_1 。

计算 S_2

我们有：

我们将 S_2 分成 S_{21} 与 S_{22} 两部分：

其中：

计算 U

由 $S_2 = S_{21} + S_{22}$ 可得，因此：

因此：

因此：

因为有 $\frac{1}{2}$ ，所以我们可以预处理出所有的 $\frac{1}{2}$ ，这样我们就可以在 $O(\sqrt{n})$ 的时间复杂度内计算出 $\frac{1}{2}$ 。

计算 V

对于计算 V 的和式中的每一项，我们都有 $\frac{1}{2}$ 。因此：

所以 V 可以被表示为：

其中：

预处理出 $\frac{1}{2}$ 我们就可以在 $O(\sqrt{n})$ 的时间复杂度内计算出 V 。

考虑我们如何加速计算 V 的过程。我们可以把 $\frac{1}{2}$ 的贡献拆分成若干个 $\frac{1}{2}$ 为定值的区间上，这样就只需要计算出每一个区间的长度和从一个区间到下一个区间的 $\frac{1}{2}$ 的改变量。

更准确的说，我们首先将 $\frac{1}{2}$ 分成两个部分，将 $\frac{1}{2}$ 这个复杂的条件简化：

接着我们把这个式子改写为：

其中：

计算 W_1 与 W_2

计算这两个值需要计算满足 $\frac{1}{2}$ 的 $\frac{1}{2}$ 的值。可以在区间 $\frac{1}{2}$ 分块筛出。在每个块中我们对于所有满足条件的 $\frac{1}{2}$ 都累加 $\frac{1}{2}$ 。

计算 W_3

对于每个 $\frac{1}{2}$ ，我们把 $\frac{1}{2}$ 分成若干个区间，每个区间都满足它们的 $\frac{1}{2}$ 是定值，每个区间我们都可以 $\frac{1}{2}$ 计算它的贡献。当我们获得一个新的 $\frac{1}{2}$ 时，我们用 $\frac{1}{2}$ 的值表计算 $\frac{1}{2}$ 。以内的质数表可以给出使得 $\frac{1}{2}$ 成立的 $\frac{1}{2}$ 。以此类推使得 $\frac{1}{2}$ 变化的下一个 $\frac{1}{2}$ 的值。

计算 W_4

相比于 $\frac{1}{2}$ ， $\frac{1}{2}$ 中 $\frac{1}{2}$ 更小，所以 $\frac{1}{2}$ 改变得更快。这时候再按照计算 $\frac{1}{2}$ 的方法计算 $\frac{1}{2}$ 就显得没有任何优势。于是我们直接暴力枚举数对 $\frac{1}{2}$ 来计算 $\frac{1}{2}$ 。

计算 W_5

我们像计算 $\frac{1}{2}$ 那样来计算 $\frac{1}{2}$ 。

计算 S_3

我们使用所有小于 $\sqrt{y}$ 的素数一次筛出区间 $[x, x+y]$ 。当我们的筛法进行到 $\sqrt{y}$ 的时候，我们算出了所有 $\leq \sqrt{y}$ 满足没有平方因子并且 $\leq y$ 的值。这个筛法是分块进行的，我们在筛选间隔中维护一个二叉树，以实时维护所有素数筛选到给定素数后的中间结果。这样我们就可以只用 $O(\sqrt{y})$ 的时间复杂度求出在筛法进行到某一个值的时候没有被筛到的数的数量。

算法的时空复杂度

时空复杂度被如下 个过程影响：

1. 计算 $P_1(x, y)$ ；
2. 计算 $P_2(x, y)$ ；
3. 计算 S_3 。

计算 $P_2(x, y)$ 的复杂度

我们已经知道了这个过程的时间复杂度为 $O(\sqrt{y})$ ，空间复杂度为 $O(\sqrt{y})$ 。

计算 W_1, W_2, W_3, W_4, W_5 的复杂度

计算 W_1 所进行的块长度为 $\sqrt{y}$ 的筛的时间按复杂度为 $O(\sqrt{y})$ ，空间复杂度为 $O(\sqrt{y})$ 。

计算 W_2 所需的时间复杂度为：

计算 W_3 的时间复杂度为：

因此，计算 W_4 的时间复杂度为：

计算 W_5 的时间复杂度为：

计算 S_3 的时间复杂度为：

计算 S_3 的复杂度

对于预处理：由于要快速查询 $\sqrt{y}$ 的值，我们没办法用普通的筛法 求出，而是要维护一个数据结构使得每次查询的时间复杂度是 $O(1)$ ，因此时间复杂度为 $O(\sqrt{y})$ 。

对于求和：对于计算 S_3 和式中的每一项，我们查询上述数据结构，一共 $O(\sqrt{y})$ 次查询。我们还需要计算和式的项数，即二叉树中叶子的个数。所有叶子的形式均为 $[x, x+y]$ ，其中 $x \leq y$ 。因此，叶子的数目是 $O(\sqrt{y})$ 级别的。所以计算 S_3 的总时间复杂度为：

总复杂度

这个算法的空间复杂度为 $O(\sqrt{y})$ ，时间复杂度为：

我们取 $\sqrt{y}$ ，就有最优时间复杂度为 $O(\sqrt{y})$ ，空间复杂度为 $O(\sqrt{y})$ 。

一些改进

我们在这里给出改进方法，以减少算法的常数，提高它的实际效率。

- 在 **终止条件** 中，我们可以用一个 ϵ 来代替 $\frac{1}{n}$ ，其中 ϵ 满足 $\epsilon < \frac{1}{n}$ 。我们可以证明这样子计算 $\phi(n)$ 的时间复杂度可以优化到：

这也为通过改变 ϵ 的值来检查计算提供了一个很好的方法。

- 为了清楚起见，我们在阐述算法的时候选择在 $\frac{1}{n}$ 处拆分来计算总和，但实际上我们只需要有 ϵ 就可以计算。我们可以利用这一点，渐近复杂性保持不变。
- 用前几个素数 p_i 预处理计算可以节省更多的时间。

参考资料与拓展阅读

本文翻译自：[Computing \$\phi\(n\)\$: the Meissel, Lehmer, Lagarias, Miller, Odlyzko method](#)

本页面最近更新：2024/4/27 21:57:06, [更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[Early0v0](#), [Peanut-Tang](#), [1196131597](#), [Alpacabla](#), [CCXXXI](#), [Enter-tainer](#), [GHLinZhengyu](#), [Great-designer](#), [lly2009](#), [megakite](#), [Menci](#), [Tiphereth-A](#), [Vxlimo](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用